

SmartCare v0.2 - Requirements Specification Template

1. Problem and Scope

Problem

SmartCare Community Clinic currently suffers from inefficient administrative workflows and scheduling risks due to manual or poorly coordinated appointment tracking.

- **The Core Issue:** Booking conflicts, untracked status changes, and lost cancellations create problems for receptionists, leading to schedule delays for practitioners, and cause frustration for patients.
- **The Root Cause:** Lack of a centralized, rule-enforced system that validates inputs, maintains active collision checks, and preserves historical booking data.
- **The Solution:** A lightweight, reliable Python-based domain model (Patient, Practitioner, Appointment) that enforces business constraints, controls state transitions, and guarantees scheduling integrity without unnecessary system complexity.

Scope

- **Patient & Practitioner Management:**
 - Registering and managing patient contact details and medical history.
 - Managing practitioner profiles, assigned room numbers, and specialty details.
- **Appointment Lifecycle & State Machine:**
 - Scheduling new bookings connecting one Patient to one Practitioner.
 - Strictly enforcing state transitions (Scheduled -> Arrived -> In Progress -> Completed or Cancelled).
- **Double-Booking Prevention:**
 - Automatically checking practitioner's availability across active bookings before confirming a new slot.
- **Cancellation & Audit Integrity:**
 - Mandating a non-empty cancellation reason-for text whenever an appointment is cancelled.
 - Retaining cancelled and completed appointments as historical objects in memory rather than hard-deleting them.
- **Schedule Querying:**
 - Retrieving daily schedules for specific practitioners and appointment histories for specific patients.

2. Stakeholders

Stakeholder	Need	Evidence
Patients	Reliable appointment booking with no scheduling overlaps or unexpected wait times.	The case study requirement that the system must prevent double-booking collisions so patients are guaranteed their allocated time slot.
Practitioners	Accurate, real-time view of their daily schedule to manage patient consultations effectively throughout the day.	The case study requiring the system to generate a daily schedule query showing practitioner availability.

Receptionists	A clear, rule-enforced booking tool that prevents accidental scheduling errors while handling cancellations easily.	The administrative workflow requirements mandating status transitions (Scheduled-> Arrived -> In Progress-> Completed) and enforcing a mandatory cancellation reason.
Practice Management	Complete historical records and audit trails of all patient visits and cancelled appointments.	Case study which implies that cancelled appointments must remain as objects in history rather than being deleted from the system.
Software Developers	Clean, modular, and independently testable business logic that enforces data validity before saving records.	The explicit architectural constraint requiring the classes (Patient, Practitioner, Appointment) to contain type hints, input validation guardrails, and isolated testability.

3. Functional Requirements

FR-01: The system shall allow administrative staff to register new patients with mandatory attributes: Patient ID, Full Name, Mobile, DoB, and Residential Address.

FR-02: The system shall allow administrative staff to update a patient's contact details (mobile and address) while preserving their original Patient ID and historical records.

FR-03: The system shall maintain healthcare practitioner (doctor) profiles including Practitioner ID, Doctor Name, Medical Specialty, and Assigned Room Number.

FR-04: The system shall allow staff to create a new appointment connecting exactly one registered Patient to exactly one registered Practitioner at a certain date and time slot.

FR-05: The system shall reject any new appointment booking attempt if the assigned practitioner already has an active appointment at the same date and time.

FR-06: The system shall strictly enforce appointment state lifecycle transitions: Scheduled -> Arrived -> In Progress -> Completed, preventing any out-of-order state modifications.

FR-07: The system shall require a non-empty text reason whenever an appointment status is changed to 'Cancelled'.

FR-08: The system shall retain all completed and cancelled appointments as historical records in memory.

FR-09: The system shall allow staff and practitioners to query and display all appointments assigned to a specific practitioner for a given date (YYYY-MM-DD).

FR-10: The system shall allow staff to retrieve the complete past and upcoming appointment history linked to a specific Patient ID.

FR-11: _____

FR-12: _____

4. Non-Functional Requirements

NFR-01: The system shall validate all input parameters upon object instantiation, raising explicit domain exceptions (or ValueErrors) for missing mandatory fields or invalid status transitions.

NFR-02: The core business and domain logic (Patient, Practitioner, Appointment) shall be structured to execute and undergo automated unit testing independently of databases, graphical user interfaces, or external frameworks.

NFR-03: Domain classes shall enforce object encapsulation by protecting internal attributes with read-only properties (@property).

NFR-04: Collision checks and daily schedule queries shall execute in under 100 milliseconds for in-memory datasets up to 1,000 active appointments.

NFR-05: The domain engine shall be fully decoupled from persistence and notification layers, allowing future integration with SQL databases, web APIs, or messaging services without modifying core business rules.

NFR-06: _____

5. User Stories

US-01: As a **Receptionist**, I want to **register a new patient with their full personal and contact details**, so that we **have an accurate, validated patient record on file for future bookings**.

US-02: As a **Receptionist**, I want to **schedule an appointment connecting a patient to a practitioner for a specific date and time**, so that **the patient receives a reserved consultation slot**.

US-03: As a **Receptionist**, I want **the system to automatically block bookings if a practitioner is already busy at that time slot**, so that **we prevent double-booking errors and avoid schedule delays**.

US-04: As a **Practitioner**, I want **the system to track appointment progress from arrival to completion**, so that I can **manage my consultation workflow and monitor patient flow through the clinic**.

US-05: As a **Practice Manager**, I want **the system to require a text reason whenever an appointment is cancelled**, so that **we maintain a clear audit trail and understand why bookings are being cancelled**.

US-06: As a **Practitioner**, I want to **view my complete schedule for any given day**, so that **I can prepare for upcoming consultations and manage my daily workload efficiently**.

6. Acceptance Criteria

GIVEN Practitioner **Dr Jones** already has an active appointment (Scheduled) on 2026-10-15 at 09:00,
WHEN a receptionist attempts to book a new appointment for Patient **John Doe** with Practitioner **Dr Jones** at 2026-10-15 at 09:00
THEN the system rejects the booking and raises an AppointmentException stating that the practitioner is already booked.

GIVEN an active appointment A001 with a status of Scheduled.
WHEN a receptionist attempts to cancel appointment A001 without providing a cancellation reason (or providing whitespace only).
THEN the system blocks the cancellation, raises an AppointmentException, and preserves the status as Scheduled.

GIVEN an appointment A001 for Practitioner **Dr Jones** at 2026-10-15 at 09:00 has been successfully updated to Cancelled with a valid reason.
WHEN a receptionist schedules a new appointment A002 for Patient **John Doe** with Practitioner **Dr Jones** at 2026-10-15 at 09:00.

THEN the system confirms the new booking successfully and retains the original cancelled appointment A001 in system history.

7. Assumptions and Open Questions

Appointment Durations are Uniform and Fixed.

Practitioners are Always Available.

8. AI Requirements Review Record

AI suggestion	Evidence?	Decision	Reason	Verification
FR-11: The system shall allow retrieval of appointment details using an Appointment ID.	Re-evaluation of receptionist workflows where staff need to look up, verify, or modify a booking directly when a patient calls with a booking reference number	ACCEPTED	Relying solely on filtering through a practitioner's daily schedule or a patient's entire history makes single-record administrative lookups inefficient.	Write a unit test that instantiates an appointment A001, stores it in a lookup registry, and verifies that querying <code>get_appointment_by_id("A001")</code> returns the exact Appointment object with matching attributes.
FR-12: The system shall prevent creation of appointments involving unregistered patients or practitioners.	Analysis of relational multiplicity and foreign entity constraints, which require that an Appointment must connect a valid, registered Patient to a valid, registered Practitioner.	ACCEPTED	Guarantees referential integrity. Without this check, the system would allow orphaned appointments linked to non-existent IDs (e.g., Patient "P999"), causing data corruption in schedule and history queries.	Write a unit test that attempts to call <code>create_booking()</code> using an unregistered <code>patient_id</code> or <code>practitioner_id</code> , asserting that the system raises an <code>AppointmentException</code> blocking creation.